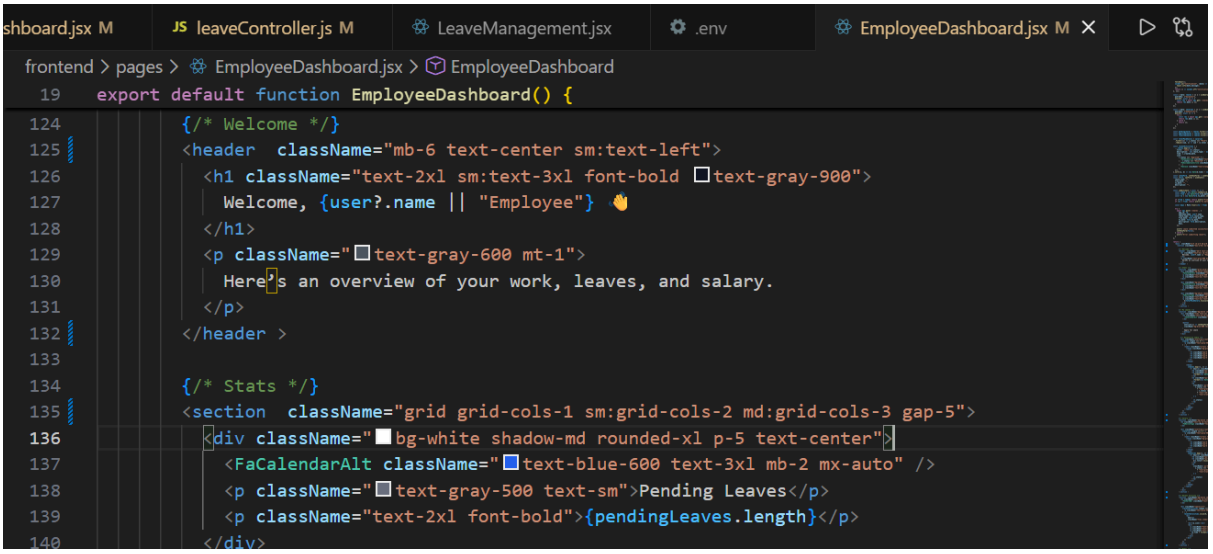
MSCIT sem 3
Employment Management System Project
Mobile and web development
ID:202412061
Name: Parmar Shenal Kamlesh

(POSTMAN screenshots available in report file)

==========

HTML, CSS and Javascript (5 Marks)

1)Use semantic elements: <header>, <Section> <nav> etc instead of <divs> wherever applicable.

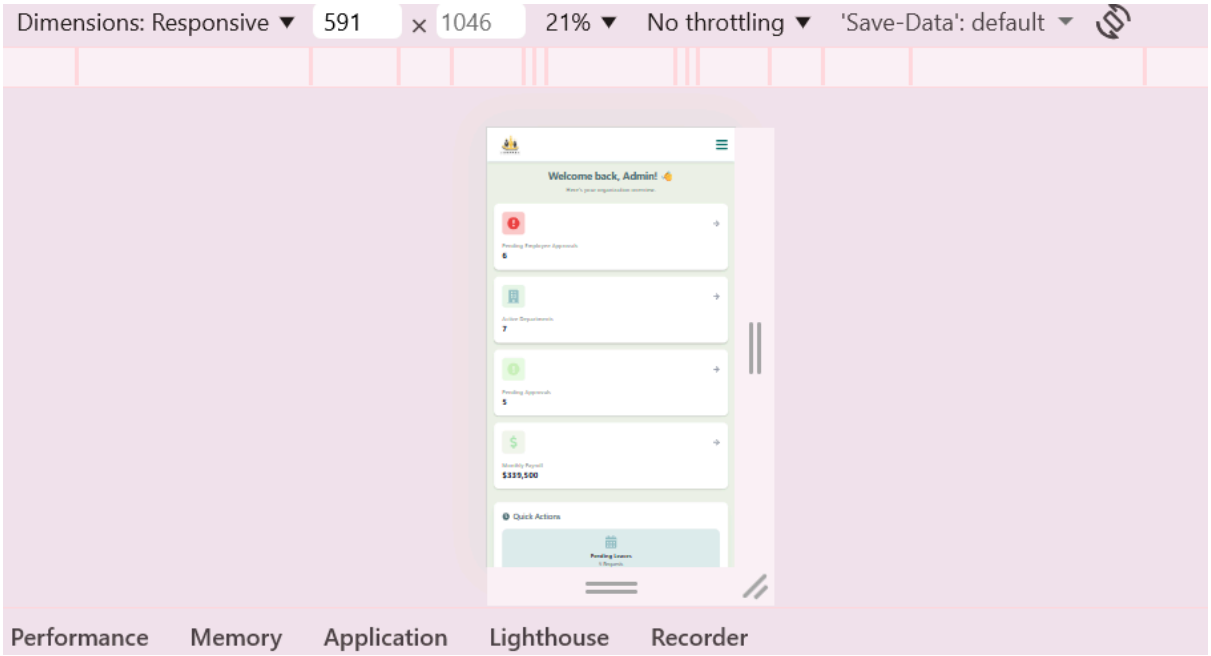


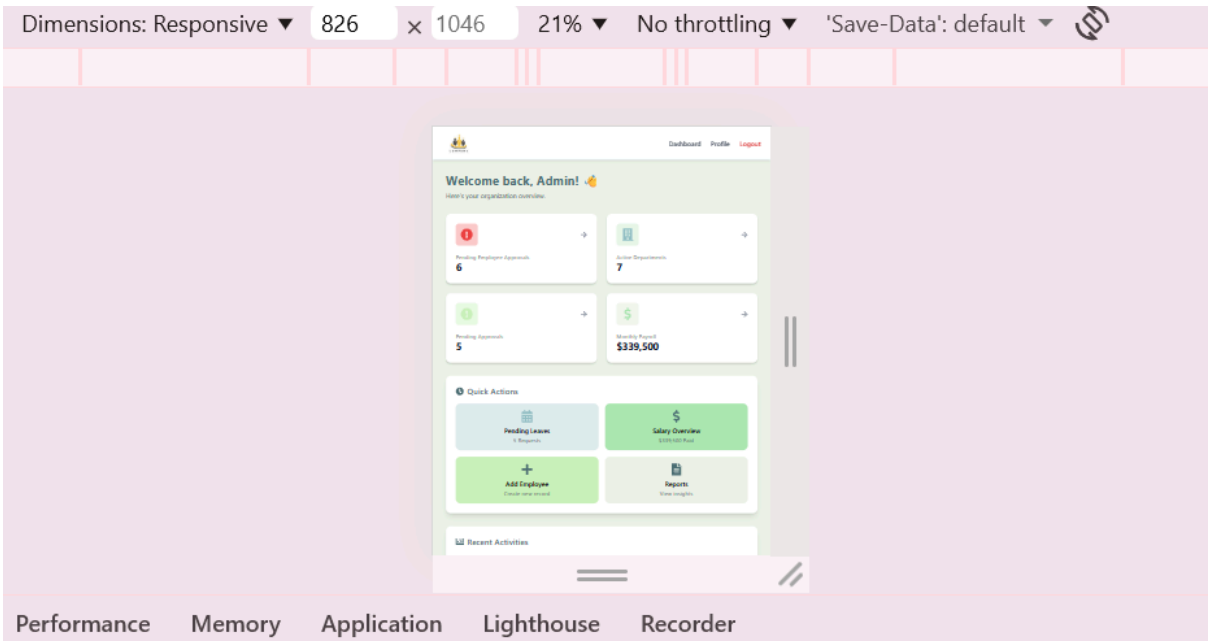
AdminDashboard.jsx M

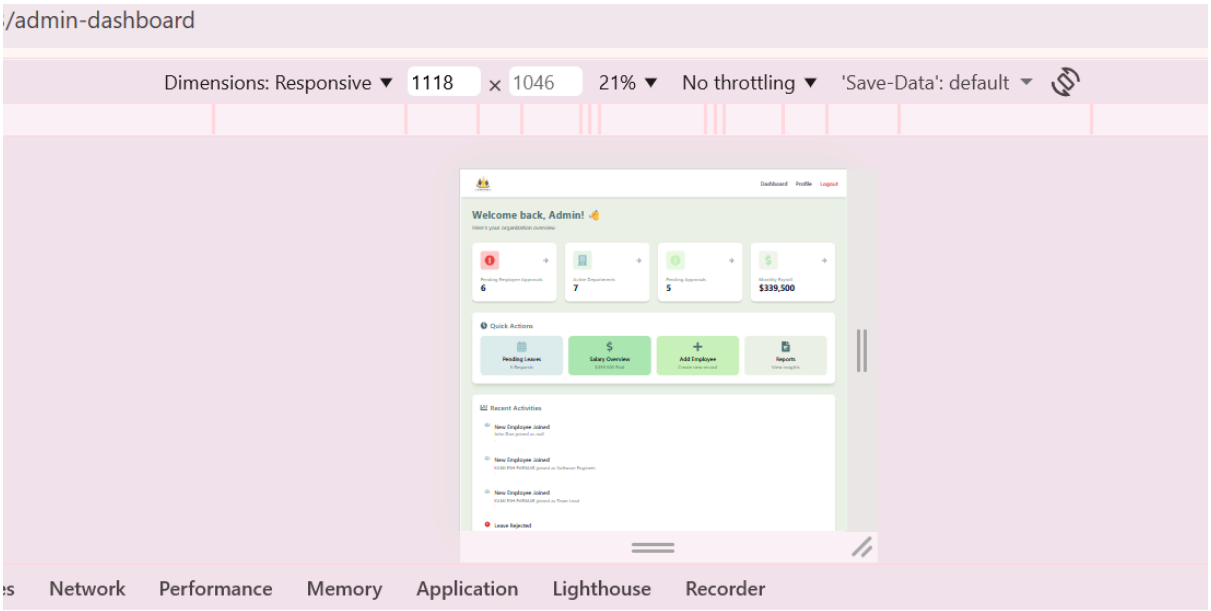
frontend > pages > AdminDashboard.jsx > AdminDashboard

```
26   export default function AdminDashboard() {
200    return (
201      <main className="p-4 sm:p-6 md:p-8 space-y-8 bg-[#ECF4E8] min-h-screen">
202        <section className="max-w-7xl mx-auto">
203
204          {/* Welcome Section */}
205          <header className="mb-6 sm:mb-8 text-center sm:text-left">
206            <h1 className="text-2xl sm:text-3xl font-bold text-[#4F6F75] mb-2">
207              Welcome back, {user?.full_name || "Admin"}! 👋
208            </h1>
209            <p className="text-gray-600 text-sm sm:text-base">
210              Here's your organization overview.
211            </p>
212          </header>
213
214          {/* Stats Section */}
215          <section  className="
216            grid
217            grid-cols-1
218            sm:grid-cols-2
219            lg:grid-cols-4
220            gap-4 sm:gap-6
221            mb-8
222          ">
223            {stats.map((stat, i) => (
224              <Link
225                to={stat.link}
```

2)should support responsive web design

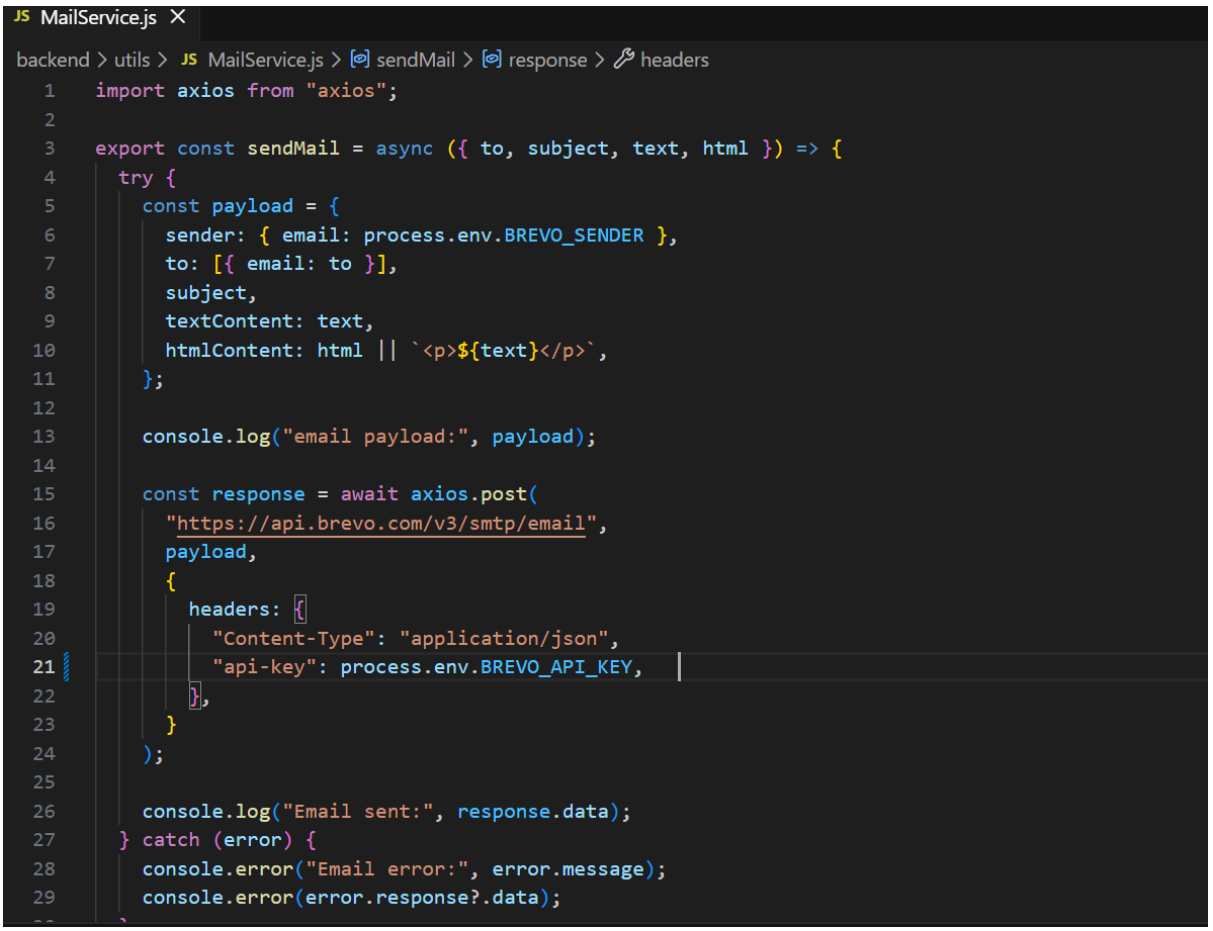






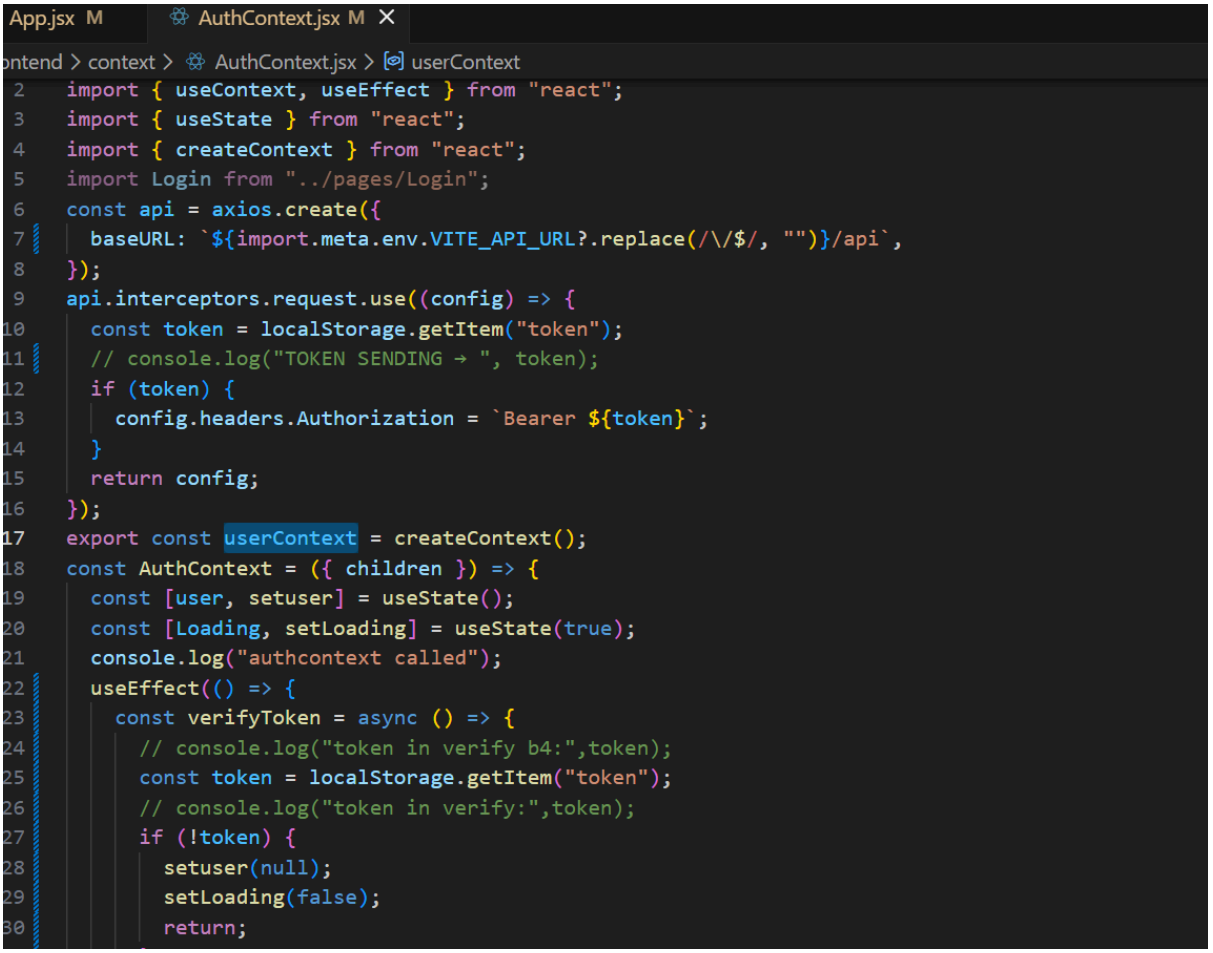
3. Tailwind CSS should be used for styling-used everywhere this only

4. Use atleast 1 thrid party Web API in your project.:brevo for email



React:

1. Use function components preferably
2. Use fetch API to make HTTP requests to server side
3. Use atleast these hooks in your project: useState, useEffect, useRef and useContext —---(used in many files)



4. Use ReactRouterDOM for navigation


frontend > src > main.jsx > ...
1 import { StrictMode } from "react";
2 import { createRoot } from "react-dom/client";
3 import "./index.css";
4 import App from "./App.jsx";
5 import { QueryClient, QueryClientProvider } from "@tar
6 import AuthContext from "../context/AuthContext.jsx";
7 import { BrowserRouter } from "react-router-dom";
8
9 const queryClient = new QueryClient();
10
11 createRoot(document.getElementById("root")).render(
12 <StrictMode>
13 <QueryClientProvider client={queryClient}>
14 <BrowserRouter>
15 <AuthContext>
16 <App />
17 </AuthContext>
18 </BrowserRouter>
19 </QueryClientProvider>
20 </StrictMode>
21);
22

```jsx
    <Routes>
      <Route path="/" element={<Login />}></Route>
      <Route path="/login" element={<Login />}></Route>
      <Route path="/reports" element={<Reports />}></Route>
      <Route
        path="/admin-dashboard"
        element={
          <ProtectedRoute role="admin">
            <AdminDashboard />
          </ProtectedRoute>
        }
      />

      <Route
        path="/employee-dashboard"
        element={
          <ProtectedRoute role="employee">
            <EmployeeDashboard />
          </ProtectedRoute>
        }
      />

      <Route
        path="/employeeManagement"
        element={<EmployeeManagement />}
      ></Route>
```

B. Node and ExpressJS Backend (25 Marks)

Use development dependencies: morgan nodemon cors

```jsx
app.use(cors({
  origin: function (origin, callback) {
    if (!origin) return callback(null, true);
    if (allowedOrigins.includes(origin)) return callback(null, true);
    return callback(new Error(`CORS blocked for origin: ${origin}`), false);
  },
  credentials: true,
  methods: ["GET","POST","PUT","PATCH","DELETE","OPTIONS"],
  allowedHeaders: ["Content-Type","Authorization"]
}));

app.get("/test-email", async (req, res) => {
  await sendMail({
    to: "shenal5420@gmail.com",
    subject: "Test email",
    text: "Hello from Brevo!",
  });
  res.send("Email sent");
});

app.use(morgan("dev"));
app.use(express.json());
```

2. Structure URLs with clear endpoints (e.g., /api/users/:id).

App.jsx M | server.js M | leaveRoutes.js M

ackend > routes > leaveRoutes.js > ...

```
import express from "express";
import {
  createLeave,
  getLeaves,
  getLeaveById,
  getMyLeaves,
  updateLeave,
  deleteLeave,
  updateLeaveStatus,
} from "../controller/leaveController.js";

const router = express.Router();

router.post("/", createLeave);
router.get("/", getLeaves);
router.get("/:id", getLeaveById);
router.get("/my-leaves/:id", getMyLeaves);
router.put("/:id", updateLeaveStatus);
router.patch("/:id", updateLeave);
router.delete("/:id", deleteLeave);

export default router;
```

3. Each API endpoint should represent a unique resource. Use route parameters to identify a resource

4. Implement proper status codes for responses (e.g., 200 for success, 404 for not found, 500
for server errors).

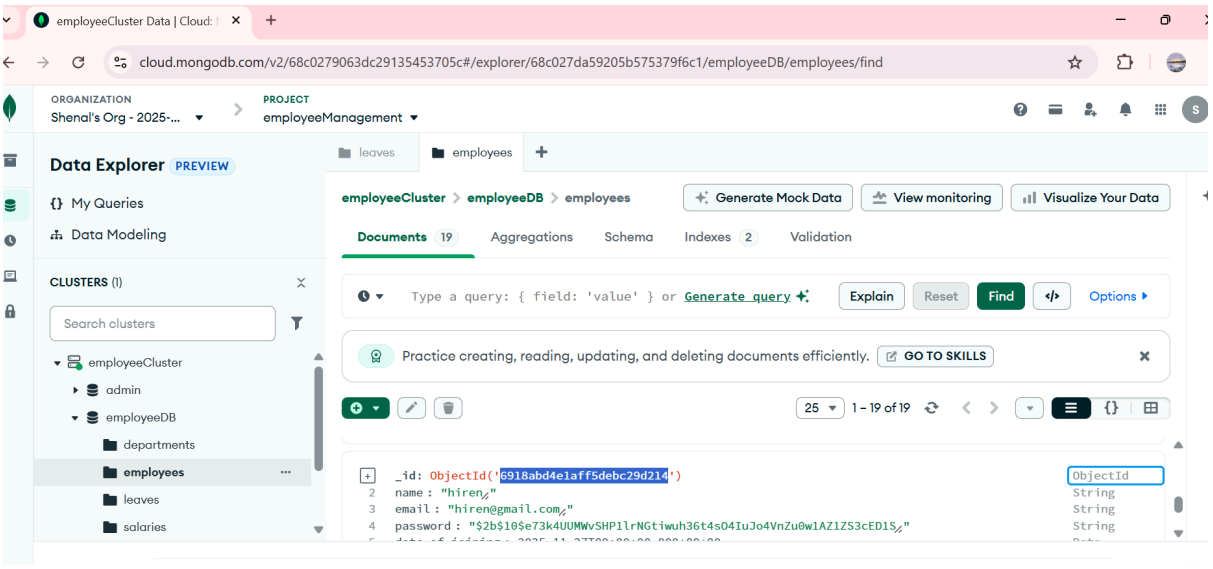
```
if (!isMatch)
  return res.status(401).json({ message: "Invalid credentials" });

const token = jwt.sign(
  { id: user._id, email: user.email, role },
  process.env.JWT_SECRET,
  { expiresIn: "7d" }
);
```

```
res.status(200).json({
  success: true,
  message: "Login successful",
  token,
  role: user.role,
  user: {
    name: user.name || user.full_name,
    email: user.email,
    id: user._id,
```

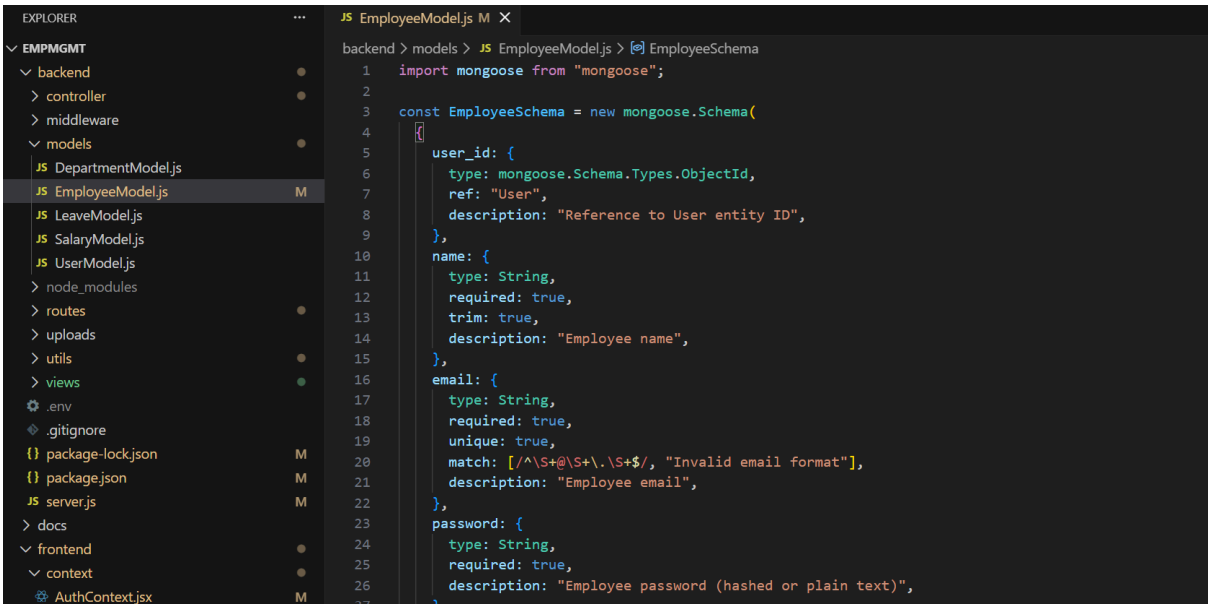
```
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

export const getCurrentUser = async (req, res) => {
  try {
    const authHeader = req.headers.authorization;
    if (!authHeader)
      return res.status(401).json({ message: "No token provided" });
```

6. Save persistent data using MongoDB and Mongoose

```
mongoose
  .connect(process.env.MONGO_URI)
  .then(() => {
    server.listen(PORT, () => console.log(`Backend running on port ${PORT}`));
  })
  .catch((err) => console.error(err));
```



7. Define schemas in the /models directory.



8. Use CRUD (atleast one each) operations.(above route ss)

9. Use jsonwebtoken for authentication

10. Use express.static() and express.json() middleware

```
app.use(morgan("dev"));
app.use(express.json());
app.use(express.static());

// ROUTES
app.use("/api/leaves", leaveRoutes);
app.use("/api/salaries", salaryRoutes);
app.use("/api/users", userRoutes);
app.use("/api/employees", empRoutes);
app.use("/api/departments", deptRoutes);
```

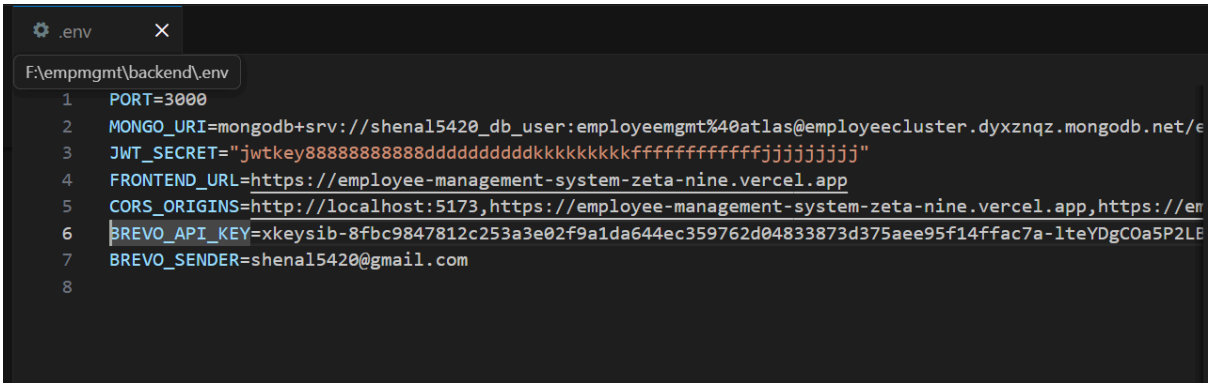
11. Write atleast one custom middleware function and use the middleware stack/chaining

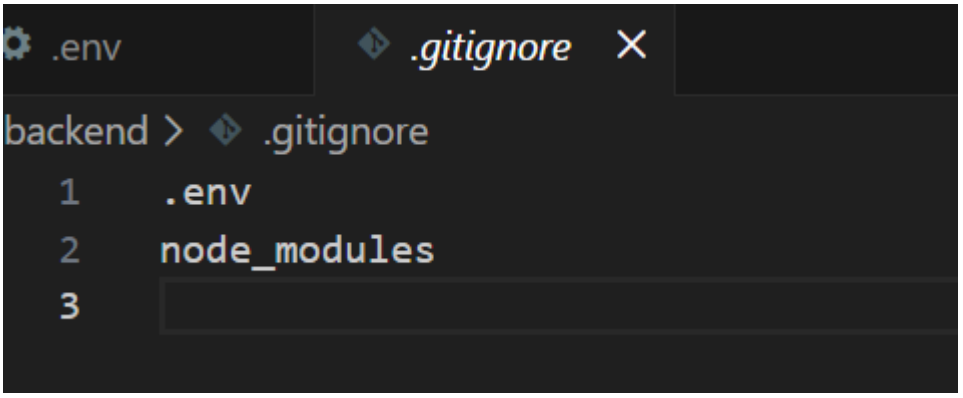
authRoutes.js

backend > routes > authRoutes.js > ...

```
1   // routes/authRoutes.js
2   import express from "express";
3   import {verifyUser} from "../middleware/verifyUser.js";
4
5   const router = express.Router();
6
7   router.get("/verify", verifyUser, (req, res) => {
8     res.json({
9       success: true,
10      user: req.user,
11      role:req.user.role,// this is the user decoded from the token
12    });
13  });
14
15
16  export default router;
17
```

12. Store sensitive data (API keys, database URIs) in environment variables using a .env file, and
add this file to .gitignore.





13. Show the server side rendering of few pages using EJS.

C. Source files and documentation (5 Marks)

1. The source code should be put on your Github account and share the link to clone the
repository. Maintain a README.md file with setup instructions, instructions to run the project, project structure, and API endpoint descriptions.

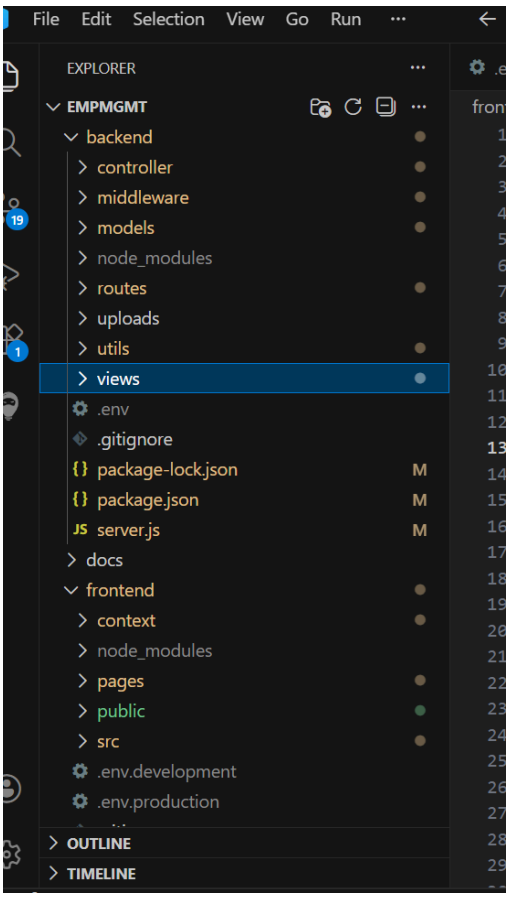
Links:

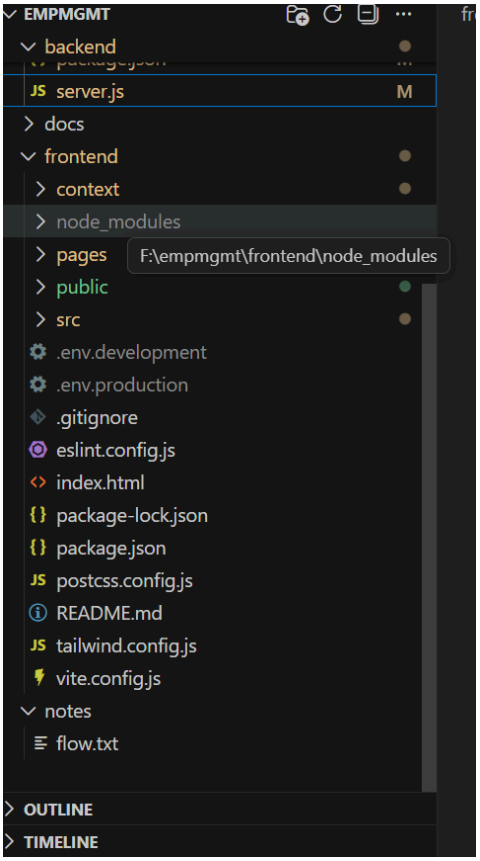
Github link: https://github.com/shenal-parmar/Employee_Management_System

Clone link :git clone
https://github.com/shenal-parmar/Employee_Management_System.git

Deployment link : https://employee-management-system-zeta-nine.vercel.app

Folder Structure 👍


File
Edit
Selection
View
Go
Run
EXPLORER
EMPMGMT
backend
controller
middleware
models
node_modules
routes
uploads
utils
views
.env
.gitignore
package-lock.json
M
package.json
M
server.js
M
docs
frontend
context
node_modules
pages
public
src
.env.development
.env.production
OUTLINE
TIMELINE


EMPMGMT
backend
server.js
M
docs
frontend
context
node_modules
pages
F:\empmgmt\frontend\node_modules
public
src
.env.development
.env.production
.gitignore
eslint.config.js
index.html
package-lock.json
package.json
postcss.config.js
README.md
tailwind.config.js
vite.config.js
notes
flow.txt
OUTLINE
TIMELINE